

🔗 Architecture Logicielle & Déploiement – CALENDRIA

Ce document décrit l'architecture multi-couches de l'application Calendria, les choix technologiques et les bonnes pratiques implémentées, conformément aux exigences du Jalon 4 (Chapitres VII et VIII).

🔗 1. Architecture N-Tiers (Déploiement Physique)

L'application suit une **architecture 3-tiers stricte**, où chaque couche a une responsabilité unique et communique via des protocoles standards.

🔗 Tier 1 : Présentation (Client)

- **Navigateur Web / Mobile** : Charge la Single Page Application (SPA) React compilée par Vite.
- Composé de fichiers statiques (HTML, CSS, JS) qui s'exécutent côté client.
- Responsable de l'affichage (UI), du routage côté client (`react-router-dom`) et de l'expérience utilisateur (UX).

🔗 Tier 2 : Logique Applicative (Backend API)

- **Serveur Web + PHP (Symfony 6.4)** : Reçoit les requêtes HTTP JSON du Tier 1 (ou des webhooks externes comme WhatsApp).
- Il s'agit d'une API **RESTful Stateless** : le serveur ne conserve aucune session client en mémoire, l'authentification se fait via un token JWT à chaque requête.
- Configure les en-têtes CORS (via `NelmioCorsBundle`) pour sécuriser les appels depuis le Frontend React.

🔗 Tier 3 : Accès aux Données (SGBD)

- **PostgreSQL 15** : Base de données relationnelle stockant les données persistantes.
- Protégé du réseau externe, accessible uniquement par le Tier 2 (Backend Symfony) via TCP/IP sur le port interne 5432.

🔗 Intégration Continue (DevOps / Docker)

Le projet est entièrement conteneurisé grâce à **Docker Compose**. Cela garantit que les environnements de développement, de test et de production sont identiques :

- `frontend` : Conteneur Node.js pour le développement Vite.
- `database` : Conteneur de la BD officielle PostgreSQL.
- `backend` : Conteneur PHP (avec PDO_PGSQL).

🔗 2. Architecture Logique (Backend Symfony)

Le backend Symfony suit le pattern architectural **MVC étendu vers une architecture hexagonale (ou orientée services)**, essentielle pour éviter les *Fat Controllers* (contrôleurs surchargés).

🔗 A. Séparation en Couches (Pattern MVC)

1. **Routing & Controllers (Couche HTTP)** :

- Point d'entrée de la requête HTTP.
- Désérialisent les données REST (JSON), appellent les services métier adéquats, et retournent des réponses HTTP (ex: 201 Created, 400 Bad Request).
- Outil principal : **API Platform** génère automatiquement ces endpoints pour le CRUD de base (GET, POST, PUT, DELETE) sur nos entités Doctrine.

2. Services (Couche Métier / Domain) :

- Contient la "vraie" logique métier de l'application.
- C'est ici que résident les règles métier complexes : algorithmes de disponibilité (`DisponibiliteService`), orchestration des webhooks chatbot (`ChatbotService`), ou intégration d'API tierces (`NotificationService`).
- Sont indépendants du contexte HTTP.

3. Entités (Couche Modèle) :

- POJO (Plain Old PHP Objects) représentant les concepts métier (ex: `Client` , `Reservation`).
- Ne contiennent aucune logique d'infrastructure, uniquement des propriétés et des getters/setters.

4. Repositories (Couche Accès Données / Persistance) :

- Le *Data Access Layer*, géré via **Doctrine ORM**.
- Isole la logique SQL/DQL. Si on veut récupérer "les réservations du jour", cette requête complexe réside dans le `ReservationRepository` , pas dans un contrôleur.

🔗 B. Principes de Conformité (SOLID)

Notre conception cible les principes SOLID :

- **SRP (Single Responsibility Principle)** : La séparation Controller (HTTP), Service (Métier), Repository (BDD) assure qu'une classe n'a qu'une seule raison de changer.
- **OCP (Open/Closed Principle)** : L'utilisation d'API Platform avec des `DataProviders` et `DataPersisters` personnalisés permet d'étendre le comportement de l'API sans modifier le cœur du package.
- **DIP (Dependency Inversion Principle)** : Les contrôleurs s'appuient sur l'Injection de Dépendances de Symfony pour recevoir des instances de services. Au lieu d'instancier un service manuellement (`$service = new Service()`), le container Symfony l'injecte dans le constructeur.

🔗 3. Composants Externes et Bibliothèques

Afin de ne pas réinventer la roue, l'architecture s'appuie sur des composants standards robustes :

🔗 Back-end (Bundles Symfony)

- **API Platform** (`api_platform`) : Standardise la création de l'API RESTful. Fournit la pagination, le filtrage, la documentation OpenAPI/Swagger native et la sérialisation (Groupes JSON).
- **Lexik JWT Authentication** (`lexik_jwt_authentication`) : Gère la création (Login) et la validation des tokens JSON Web Token pour sécuriser l'API (Stateless).
- **Nelmio CORS** (`nelmio_cors`) : Gère la politique Cors-Origin pour permettre au navigateur d'interroger l'API depuis le port 5173 du frontend.
- **Doctrine ORM** : Fait le pont entre les objets PHP (`Entities`) et la base de données PostgreSQL (génération des migrations SQL).

🔗 Services Tiers (APIs)

- **Twilio API** : Utilisée dans les processus asynchrones pour expédier les notifications SMS (rappels et confirmations).
- **OpenAI API (ChatGPT)** : Le "cerveau" NLP (Natural Language Processing) du chatbot. Interprète les intentions des clients (ex: "Je veux une table") et extrait les variables (Date, heure, personnes) à partir de texte libre.